

Отчёт по лабораторной работе №7

Имитационное моделирование

бахи сиди али темассини

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
2.1	Подготовка рабочего проекта	7
2.2	Проверка проектного окружения	7
2.3	Выполнение модели M/M/c	8
2.4	Преобразование модели M/M/c в литературный стиль	8
2.5	Выполнение модели M/M/c в Jupyter Notebook	9
2.6	Выполнение модели Росса	9
2.7	Анализ результатов моделирования Росса	10
2.8	Преобразование модели Росса в литературный стиль	11
2.9	Выполнение модели Росса в Jupyter Notebook	11
3	Модель M/M/c	12
3.1	Подключение пакетов	12
3.2	Параметры модели	13
3.3	Функция поведения клиента	13
3.4	Запуск модели	14
3.5	Аналитические формулы	14
3.6	График аналитических характеристик	15
3.7	Основная функция	16
4	Модель Росса (исправленная версия)	18
4.1	Параметры модели	19
4.2	Структура для мониторинга	19
4.3	Состояние системы	20
4.4	Статистика ремонта (ручной счётчик очереди)	20
4.5	Поведение одной машины	21
4.6	Инициализация системы	23
4.7	Запуск одного прогона	24
4.8	Прогон для разных параметров	24
4.9	Аналитическая оценка	27
4.10	Построение графиков для одиночного прогона	27
4.11	Построение тепловой карты результатов	29
4.12	Сравнение с аналитикой	31
4.13	Анализ загрузки ремонтников	33

4.14 Основная функция	34
5 Выводы	39

Список иллюстраций

2.1	Создание рабочего проекта lab07 и инициализация окружения Julia .	7
2.2	Проверка установленных пакетов проектного окружения Julia	8
2.3	Выполнение модели M/M/2 и расчёт аналитических характеристик .	8
2.4	Генерация Julia-скрипта, документа Quarto и Jupyter Notebook для модели M/M/c	9
2.5	Выполнение литературной версии модели M/M/c в Jupyter Notebook .	9
2.6	Выполнение одиночного и массовых прогонов модели Росса	10
2.7	Сравнение результатов симуляции и аналитических значений модели Росса	10
2.8	Генерация Julia-скрипта, документа Quarto и Jupyter Notebook для модели Росса	11
2.9	Выполнение литературной версии модели Росса в Jupyter Notebook .	11

Список таблиц

1 Цель работы

Изучить принципы дискретно-событийного моделирования на примере систем массового обслуживания. Реализовать и исследовать модель $M/M/c$ и модель Росса, выполнить моделирование, рассчитать основные характеристики систем и сравнить результаты симуляции с аналитическими значениями.

2 Выполнение лабораторной работы

2.1 Подготовка рабочего проекта

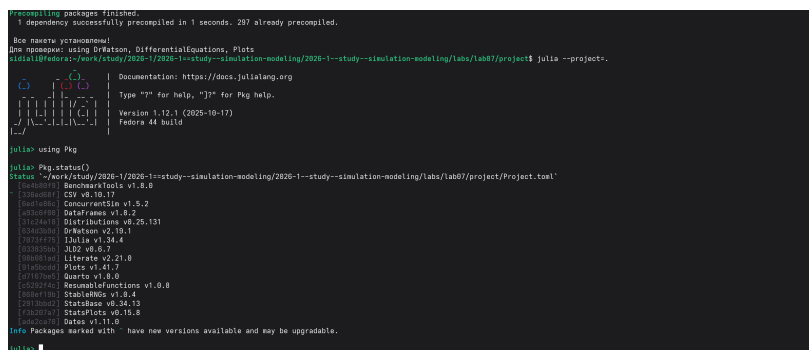
Для выполнения лабораторной работы был создан проект lab07/project с помощью скрипта `setup_project.jl`. При запуске скрипта сформировано проектное окружение Julia и установлена зависимость DrWatson (рис. 2.1).

```
julia@fedora:~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab7$ ls
environment_report setup_project.jl
julia@fedora:~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab7$ cat setup_project.jl
cd /usr/bin/new_julia
# setup_project.jl
using Pkg
Pkg.add("DrWatson")
using DrWatson
# Create new envs
project_name = "project"
initialize_project(project_name; subdirs="std ali", git=false)
println("Project name: ", project_name)
julia@fedora:~/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab7$ julia setup_project.jl
Resolving package versions...
Project No packages added to or removed from "/julia/environments/v1.12/project.toml".
Manifest No packages added to or removed from "/julia/environments/v1.12/Manifest.toml".
Activating new project at "/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab7/project"
Resolving package versions...
Updating "/work/study/2026-1/study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab7/project/Project.toml"
[534639d] + DrWatson v2.1.1
[43b5a4e] + ChunkCodec12Lib v1.1.0
[534639d] + ChunkCodec12Lib v1.1.0
[534639d] + DrWatson v2.1.1
[534639d] + Field v0.9.0
[0f00010] + HashArrayMappedTries v0.2.8
[03030a0] + JLD2 v0.8.7
[03030a0] + JLD2Wrappers v1.8.8
[19140d7] + MacroTools v0.5.10
[ba00000] + OrderedCollections v2.8.1
[ae77000] + PrecompileTools v1.2.4
[7100000] + Preferences v0.5.0
[ae57000] + Requires v1.2.1
[7400000] + ScopedValues v1.0.2
[534639d] + Serato v1.3.0
[534639d] + Serato v1.3.0
```

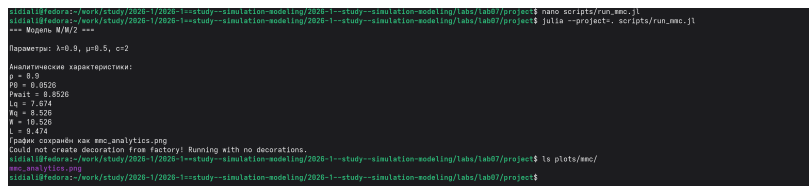
Рисунок 2.1: Создание рабочего проекта lab07 и инициализация окружения Julia

2.2 Проверка проектного окружения

Проект запущен командой `julia --project=..`. С помощью `Pkg.status()` проверено наличие установленных пакетов, включая `ConcurrentSim`, `Distributions`, `ResumableFunctions`, `StableRNGs`, `DataFrames` и `Plots` (рис. 2.2).



2.3 Выполнение модели M/M/c



2.4 Преобразование модели M/M/c в литературный стиль


```
cd /home/sergey/study/2020-1/2020-1-study-simulation-modeling/2020-1-study-simulation-modeling/labs/lab07/project$ julia --project= scripts/run_ross.jl
=====
МОДЕЛЬ РОССА
=====
Параметры по умолчанию:
N = 10 (работавшие машины)
S = 3 (ремонтники машины)
R = 2 (дефектных)
λ = 1/100.0 отказов/час (на одну машину)
μ = 1/1.0 ремонт/час (на один ремонт)

Оценочные интенсивности:
Отказы: 0.1 отказов/час
Ремонт: 2.0 ремонт/час
Состояние системы

=====
ОДИНОЧНЫЙ ПРОГОН (визуализация)
=====
Запуск симуляции с параметрами по умолчанию...
Симуляция стартует в time 78.3026293109867: no spares
Время до падения: 78.38 часов
Графики сохранены в ross_results.png

=====
МАССОВЫЕ ПРОГОНЫ (stats)
=====
(1/27) Running N=5, S=2, R=1...
Could not create decoration from factory! Running with no decorations.
(2/27) Running N=5, S=2, R=1...
(3/27) Running N=5, S=3, R=1...
(4/27) Running N=5, S=3, R=1...
(5/27) Running N=5, S=3, R=1...
(6/27) Running N=5, S=3, R=1...
(7/27) Running N=5, S=3, R=1...
(8/27) Running N=5, S=3, R=1...
(9/27) Running N=5, S=3, R=1...
(10/27) Running N=5, S=3, R=1...
(11/27) Running N=5, S=3, R=1...
(12/27) Running N=5, S=3, R=1...
```

Рисунок 2.6: Выполнение одиночного и массовых прогонов модели Росса

2.7 Анализ результатов моделирования Росса

После массовых прогонов выполнены визуализация и сравнение результатов симуляции с аналитическими значениями. Сформированы тепловые карты для $R = 1$ и $R = 2$, а также проведён анализ загрузки ремонтников. Результирующие графики сохранены в каталоге `plots/ross` (рис. 2.7).

```
=====
АНАЛИЗ ЗАГРУЗКИ РЕМОНТНИКОВ
=====
Конфигурация: N=10, S=3, R=1
-----
Средняя длина очереди: 0.6
Максимальная длина очереди: 0

Конфигурация: N=10, S=3, R=2
-----
Средняя длина очереди: 0.6
Максимальная длина очереди: 0

Конфигурация: N=10, S=3, R=3
-----
Средняя длина очереди: 0.6
Максимальная длина очереди: 0

=====
АНАЛИЗ ЗАВЕРШЕН
=====
cd /home/sergey/study/2020-1/2020-1-study-simulation-modeling/2020-1-study-simulation-modeling/labs/lab07/project$ ls plots/ross/
ross_heatmap_R1.png  ross_heatmap_R2.png  ross_results.png
cd /home/sergey/study/2020-1/2020-1-study-simulation-modeling/2020-1-study-simulation-modeling/labs/lab07/project$
```

Рисунок 2.7: Сравнение результатов симуляции и аналитических значений модели Росса

2.8 Преобразование модели Росса в литературный стиль

Для документирования модели подготовлен файл `run_ross_literary.jl`. С помощью `tangle.jl` автоматически сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.8).

```
sidali@fedora:~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ nano scripts/run_ross_literary.jl
sidali@fedora:~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$ julia --project= scripts/tangle.jl scripts/run_ross_literary.jl
[генерация из: scripts/run_ross_literary.jl]
[info: generating plain script file from "/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project/scripts/run_ross_literary.jl"]
[info: writing result to "/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project/scripts/run_ross_literary.jl"]
[info: generating markdown page from "/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project/scripts/run_ross_literary.jl"]
[info: writing result to "/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project/markdown/run_ross_literary/run_ross_literary.qmd"]
[info: generating notebook from "/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project/scripts/run_ross_literary.jl"]
[info: writing result to "/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project/notebooks/run_ross_literary/run_ross_literary.ipynb"]
[info: Notebook "/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project/notebooks/run_ross_literary/run_ross_literary.ipynb"
[готово! Все файлы созданы.]
sidali@fedora:~/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab07/project$
```

Рисунок 2.8: Генерация Julia-скрипта, документа Quarto и Jupyter Notebook для модели Росса

2.9 Выполнение модели Росса в Jupyter Notebook

Сгенерированный `run_ross_literary.ipynb` выполнен в Jupyter Notebook. В ноутбуке представлены функции анализа загрузки ремонтников и основная функция запуска модели с заданными параметрами и проверкой стационарности системы (рис. 2.9).

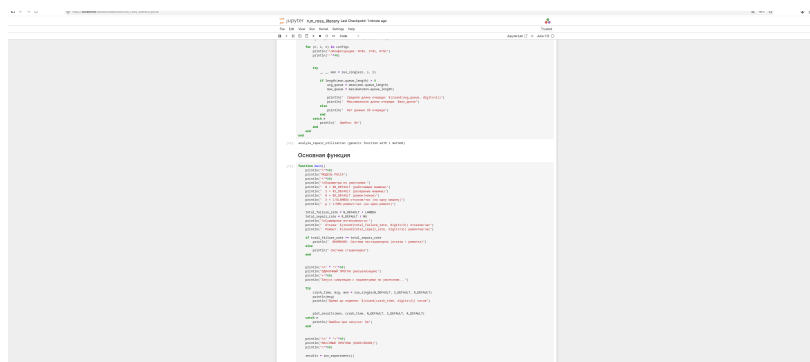


Рисунок 2.9: Выполнение литературной версии модели Росса в Jupyter Notebook

3 Модель M/M/c

Система массового обслуживания с: - пуассоновским входящим потоком (λ) - экспоненциальным временем обслуживания (μ) - с параллельными каналами

3.1 Подключение пакетов

```
using DrWatson
quickactivate("../project", "project")

using Distributions
using ConcurrentSim
using ResumableFunctions
using Random
using StableRNGs
using Plots

mkpath(datadir("mmc"))
mkpath(plotsdir("mmc"))
```

Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project`

```
"/home/sidiali/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab07/project/plots/mmc"
```

3.2 Параметры модели

```
rng = StableRNG(123)  
num_customers = 1000  
num_servers = 2  
mu = 1.0 / 2.0 # интенсивность обслуживания  
lam = 0.9      # интенсивность поступления  
  
arrival_dist = Exponential(1 / lam)  
service_dist = Exponential(1 / mu)
```

```
Exponential{Float64}(θ=2.0)
```

3.3 Функция поведения клиента

```
@resumable function customer(  
    env::Environment,  
    server::Resource,  
    id::Integer,  
    t_a::Float64,  
    d_s::Distribution,  
)  
    @yield timeout(env, t_a)  
    println("Customer $id arrived: ", now(env))  
    @yield request(server)
```

```

println("Customer $id entered service: ", now(env))
@yield timeout(env, rand(rng, d_s))
@yield release(server)
println("Customer $id exited service: ", now(env))
end

```

customer (generic function with 1 method)

3.4 Запуск модели

```

function run_mmc()
    sim = Simulation()
    server = Resource(sim, num_servers)
    arrival_time = 0.0

    for i in 1:num_customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end

    run(sim)
end

```

run_mmc (generic function with 1 method)

3.5 Аналитические формулы

```

function analytical_metrics( $\lambda$ ,  $\mu$ , c)

     $\rho$  =  $\lambda$  / (c *  $\mu$ )

    sum1 = sum((c *  $\rho$ )^n / factorial(n) for n in 0:c-1)
    sum2 = (c *  $\rho$ )^c / (factorial(c) * (1 -  $\rho$ ))
    P0 = 1 / (sum1 + sum2)

    Pwait = (c *  $\rho$ )^c / (factorial(c) * (1 -  $\rho$ )) * P0

    Lq =  $\rho$  / (1 -  $\rho$ ) * Pwait

    Wq = Lq /  $\lambda$ 
    W = Wq + 1 /  $\mu$ 
    L =  $\lambda$  * W

    return ( $\rho$ = $\rho$ , P0=P0, Pwait=Pwait, Lq=Lq, Wq=Wq, W=W, L=L)
end

```

analytical_metrics (generic function with 1 method)

3.6 График аналитических характеристик

```

function plot_analytics()
    λ_range = 0.1:0.05:1.5
    c_vals = [1, 2, 3, 4]

    plots = []
    for c in c_vals
        μ_fixed = 1.0
        Pwait_vals = [c * μ_fixed > λ ?
            analytical_metrics(λ, μ_fixed, c).Pwait : NaN for λ in λ_range]

        p = plot(λ_range, Pwait_vals,
            label="c=$c",
            xlabel="λ", ylabel="Pwait",
            title="Вероятность ожидания")
        push!(plots, p)
    end

    plot(plots..., layout=(2,2))
    savefig(plotsdir("mmc","mmc_analytics.png"))
    println("График сохранён как mmc_analytics.png")
end

```

plot_analytics (generic function with 1 method)

3.7 Основная функция

run_mmc() # раскомментировать для запуска симуляции

```

function main()

    println("=== Модель M/M/$num_servers ===\n")
    println("Параметры:  $\lambda$ =$lam,  $\mu$ =$mu, c=$num_servers")

    metrics = analytical_metrics(lam, mu, num_servers)
    println("\nАналитические характеристики:")
    println("ρ = $(round(metrics.ρ, digits=3))")
    println("P0 = $(round(metrics.P0, digits=4))")
    println("Pwait = $(round(metrics.Pwait, digits=4))")
    println("Lq = $(round(metrics.Lq, digits=3))")
    println("Wq = $(round(metrics.Wq, digits=3))")
    println("W = $(round(metrics.W, digits=3))")
    println("L = $(round(metrics.L, digits=3))")

    plot_analytics()

end

if abspath(PROGRAM_FILE) == @__FILE__
    main()
end

```

4 Модель Росса (исправленная версия)

Система с: - N работающих машин - S резервных машин - R ремонтников - экспоненциальными отказами и ремонтом

```
using DrWatson
quickactivate("../project", "project")

using Distributions
using ConcurrentSim
using ResumableFunctions
using Random
using StableRNGs
using DataFrames
using Plots
using CSV
using Statistics
using Printf          # ← ДОБАВЛЕНО для @sprintf
```

Activating project at `~/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab07/project`

4.1 Параметры модели

```
const RUNS = 20 # количество прогонов для статистики
const N_DEFAULT = 10 # работающие машины
const S_DEFAULT = 3 # резервные машины
const R_DEFAULT = 2 # количество ремонтников
const LAMBDA = 100.0 # среднее время безотказной работы (часов)
const MU = 1.0 # среднее время ремонта (часов)
const SEED = 150

mkpath(datadir("ross"))
mkpath(plotsdir("ross"))

rng = StableRNG(SEED)
failure_dist = Exponential(LAMBDA)
repair_dist = Exponential(MU)
```

```
Exponential{Float64}(θ=1.0)
```

4.2 Структура для мониторинга

```
mutable struct Monitor
    time::Vector{Float64}
    operational::Vector{Int} # работающие машины
    spare::Vector{Int} # резервные машины
    in_repair::Vector{Int} # машины в ремонте
```

```

    queue_length::Vector{Int}      # очередь на ремонт
end

Monitor() = Monitor(Float64[], Int[], Int[], Int[], Int[])

function record!(mon::Monitor, env, operational, spare, in_repair, queue_len)
    push!(mon.time, now(env))
    push!(mon.operational, operational)
    push!(mon.spare, spare)
    push!(mon.in_repair, in_repair)
    push!(mon.queue_length, queue_len)
end

```

record! (generic function with 1 method)

4.3 Состояние системы

```

mutable struct SystemState
    operational::Int
    spare::Int
    in_repair::Int
end

```

4.4 Статистика ремонта (ручной счётчик очереди)

```

mutable struct RepairStats
    busy::Int
end

```

```
    queue :: Int
end
```

4.5 Поведение одной машины

```
@resumable function machine(
    env :: Environment,
    repair_facility :: Resource,
    state :: SystemState,
    repair_stats :: RepairStats,
    mon :: Monitor,
    N :: Int,
    S :: Int,
    R :: Int
)
    while true

        @yield timeout(env, rand(rng, failure_dist))

        state.operational -= 1
        state.spare -= 1

        record!(mon, env, state.operational, state.spare,
            state.in_repair, repair_stats.queue)
```

```

if state.spare < 0
    throw(StopSimulation("System crash at time $(now(env)): no spares"))
end

if repair_stats.busy >= R
    repair_stats.queue += 1
    record!(mon, env, state.operational, state.spare,
            state.in_repair, repair_stats.queue)
end

@yield request(repair_facility)
repair_stats.busy += 1
if repair_stats.queue > 0
    repair_stats.queue -= 1
end
state.in_repair += 1

record!(mon, env, state.operational, state.spare,
        state.in_repair, repair_stats.queue)

@yield timeout(env, rand(rng, repair_dist))

@yield release(repair_facility)
repair_stats.busy -= 1

```

```

state.in_repair -= 1
state.spare += 1

if state.spare > 0 && state.operational < N
    state.operational += 1
    state.spare -= 1
end

record!(mon, env, state.operational, state.spare,
        state.in_repair, repair_stats.queue)
end
end

```

machine (generic function with 1 method)

4.6 Инициализация системы

```

function setup_system(env, repair_facility, state, repair_stats, mon, N, S, R)
    for i in 1:N
        @process machine(env, repair_facility, state, repair_stats, mon, N, S, R)
    end
    return nothing
end

```

setup_system (generic function with 1 method)

4.7 Запуск одного прогона

```
function run_single(N::Int, S::Int, R::Int)
    sim = Simulation()
    repair_facility = Resource(sim, R)
    state = SystemState(N, S, 0)
    repair_stats = RepairStats(0, 0)
    mon = Monitor()

    record!(mon, sim, state.operational, state.spare, state.in_repair, 0)

    setup_system(sim, repair_facility, state, repair_stats, mon, N, S, R)

    msg = run(sim)
    crash_time = now(sim)

    return crash_time, msg, mon
end
```

run_single (generic function with 1 method)

4.8 Прогон для разных параметров

```

function run_experiments()
    results = DataFrame(
        N=Int[], S=Int[], R=Int[],
        mean_time=Float64[], std_time=Float64[],
        min_time=Float64[], max_time=Float64[]
    )

    n_values = [5, 10, 15]
    s_values = [2, 3, 5]
    r_values = [1, 2, 3]

    total_combinations = length(n_values) * length(s_values) * length(r_values)
    current = 0

    for n in n_values
        for s in s_values
            for r in r_values
                current += 1
                println("[$current/$total_combinations] Running N=$n, S=$s, R=$r...")

                times = Float64[]

                for run_idx in 1:RUNS
                    try
                        t, _, _ = run_single(n, s, r)
                        if t > 0
                            push!(times, t)

```

```

        end
      catch e
        push!(times, 0.0)
      end
    end
  end

  times = filter(x → x > 0, times)

  if length(times) > 0
    push!(results, (
      n, s, r,
      mean(times),
      std(times),
      minimum(times),
      maximum(times)
    ))
  else
    push!(results, (n, s, r, 0.0, 0.0, 0.0, 0.0))
  end
end
end
end

return results
end

```

run_experiments (generic function with 1 method)

4.9 Аналитическая оценка

```
function analytical_mttf(N::Int, S::Int, R::Int, lambda::Float64, mu::Float64)

    λ = N / lambda
    μ = R / mu

    if λ >= μ
        return Inf
    end

    ρ = λ / μ

    if R == 1
        return (S + 1) / λ * (1 / (1 - ρ))
    else
        return (S + 1) / λ * (1 / (1 - ρ)) * (1 - ρ^R) / (1 - ρ^(R+1))
    end
end
```

analytical_mttf (generic function with 1 method)

4.10 Построение графиков для одиночного прогона

```

function plot_results(mon::Monitor, crash_time::Float64, n::Int, s::Int, r::Int)
    if length(mon.time) == 0
        @warn "Нет данных для построения графиков"
        return nothing
    end

    total_good = [mon.operational[i] + mon.spare[i] for i in 1:length(mon.time)]

    p1 = plot(mon.time, total_good,
        label="Исправные машины (всего)",
        xlabel="Время (часы)", ylabel="Количество",
        title="Динамика числа исправных машин (N=$n, S=$s, R=$r)",
        linewidth=2, color=:blue)
    vline!([crash_time], label="Падение системы", linestyle=:dash, linewidth=2, color=:red)
    hline!([n], label="Необходимо для работы", linestyle=:dot, color=:green)

    p2 = plot(mon.time, mon.operational,
        label="Работающие машины",
        xlabel="Время (часы)", ylabel="Количество",
        title="Работающие машины",
        linewidth=2, color=:blue)
    hline!([n], label="Требуется N=$n", linestyle=:dot, color=:red)

    p3 = plot(mon.time, mon.spare,
        label="Резервные машины",

```

```

        xlabel="Время (часы)", ylabel="Количество",
        title="Резерв",
        linewidth=2, color=:green)

p4 = plot(mon.time, mon.queue_length,
        label="Длина очереди на ремонт",
        xlabel="Время (часы)", ylabel="Очередь",
        title="Очередь на ремонт (R=$r ремонтников)",
        linewidth=2, color=:orange)

p = plot(p1, p2, p3, p4, layout=(4,1), size=(800, 1000))

savefig(plotsdir("ross", "ross_results.png"))
println("Графики сохранены в ross_results.png")

return p
end

```

plot_results (generic function with 1 method)

4.11 Построение тепловой карты результатов

```

function plot_heatmap(results::DataFrame)

    r1_data = filter(:R == (1), results)

```

```

if nrow(r1_data) > 0 && length(unique(r1_data.S)) > 1 && length(unique(r1_data.N)) > 1
  p1 = heatmap(
    unique(r1_data.S),
    unique(r1_data.N),
    reshape(r1_data.mean_time, length(unique(r1_data.N)), length(unique(r1_data.S)))
    xlabel="Резерв (S)", ylabel="Работающие машины (N)",
    title="Среднее время до падения (R=1)",
    color=:viridis,
    aspect_ratio=:equal
  )
  savefig(plotsdir("ross", "ross_heatmap_R1.png"))
  println("Тепловая карта для R=1 сохранена в ross_heatmap_R1.png")
end

r2_data = filter(:R == 2, results)
if nrow(r2_data) > 0 && length(unique(r2_data.S)) > 1 && length(unique(r2_data.N)) > 1
  p2 = heatmap(
    unique(r2_data.S),
    unique(r2_data.N),
    reshape(r2_data.mean_time, length(unique(r2_data.N)), length(unique(r2_data.S)))
    xlabel="Резерв (S)", ylabel="Работающие машины (N)",
    title="Среднее время до падения (R=2)",
    color=:viridis,
    aspect_ratio=:equal
  )
  savefig(plotsdir("ross", "ross_heatmap_R2.png"))
  println("Тепловая карта для R=2 сохранена в ross_heatmap_R2.png")
end

```

```
end  
end
```

plot_heatmap (generic function with 1 method)

4.12 Сравнение с аналитикой

```
function compare_with_analytical()  
    println("\n" * "="^60)  
    println("СРАВНЕНИЕ СИМУЛЯЦИИ С АНАЛИТИКОЙ")  
    println("="^60)  
  
    test_cases = [  
        (10, 2, 1), (10, 3, 1), (10, 5, 1),  
        (10, 2, 2), (10, 3, 2), (10, 5, 2),  
        (10, 2, 3), (10, 3, 3), (10, 5, 3)  
    ]  
  
    println("\nN      S      R      Симуляция      Аналитика      Отклонение      Доверительный"  
    println("-"^80)  
  
    for (n, s, r) in test_cases  
  
        sim_times = Float64[]  
        for run_idx in 1:RUNS  
            try  
                t, _, _ = run_single(n, s, r)  
                if t > 0
```

```

        push!(sim_times, t)
    end
catch

end

end

if length(sim_times) > 0
    sim_mean = mean(sim_times)
    sim_std = std(sim_times)
    sim_se = sim_std / sqrt(length(sim_times))
else
    sim_mean = 0.0
    sim_std = 0.0
    sim_se = 0.0
end

analytic = analytical_mttf(n, s, r, LAMBDA, MU)

if analytic > 0 && analytic != Inf && sim_mean > 0
    deviation = (sim_mean - analytic) / analytic * 100
    dev_str = @sprintf("%+.1f%%", deviation)
else
    dev_str = "N/A"
end

```

```

    if sim_se > 0
        ci = @sprintf("%.1f ± %.1f", sim_mean, 1.96 * sim_se)
    else
        ci = "N/A"
    end

    println(@sprintf("%-8d %-8d %-8d %10.1f %10.1f %12s %16s",
                    n, s, r, sim_mean, analytic, dev_str, ci))
end
end
end

```

compare_with_analytical (generic function with 1 method)

4.13 Анализ загрузки ремонтников

```

function analyze_repair_utilization()
    println("\n" * "="^60)
    println("АНАЛИЗ ЗАГРУЗКИ РЕМОНТНИКОВ")
    println("="^60)

    configs = [(10, 3, 1), (10, 3, 2), (10, 3, 3)]

    for (n, s, r) in configs
        println("\nКонфигурация: N=$n, S=$s, R=$r")
        println("-"^40)
    end
end

```

```

try
    _, _, mon = run_single(n, s, r)

    if length(mon.queue_length) > 0
        avg_queue = mean(mon.queue_length)
        max_queue = maximum(mon.queue_length)

        println(" Средняя длина очереди: $(round(avg_queue, digits=2))")
        println(" Максимальная длина очереди: $max_queue")
    else
        println(" Нет данных об очереди")
    end
catch e
    println(" Ошибка: $e")
end
end
end

```

analyze_repair_utilization (generic function with 1 method)

4.14 Основная функция

```

function main()
    println("="^60)
    println("МОДЕЛЬ РОССА")
    println("="^60)
    println("\nПараметры по умолчанию:")
    println(" N = $N_DEFAULT (работающие машины)")

```

```

println(" S = $S_DEFAULT (резервные машины)")
println(" R = $R_DEFAULT (ремонтников)")
println(" λ = 1/$LAMBDA отказов/час (на одну машину)")
println(" μ = 1/$MU ремонт/час (на один ремонт)")

total_failure_rate = N_DEFAULT / LAMBDA
total_repair_rate = R_DEFAULT / MU
println("\nСуммарные интенсивности:")
println(" Отказы: $(round(total_failure_rate, digits=3)) отказов/час")
println(" Ремонт: $(round(total_repair_rate, digits=3)) ремонтов/час")

if total_failure_rate >= total_repair_rate
    println(" ВНИМАНИЕ: Система нестационарна (отказы > ремонта)")
else
    println(" Система стационарна")
end

println("\n" * "="^60)
println("ОДИНОЧНЫЙ ПРОГОН (визуализация)")
println("="^60)
println("Запуск симуляции с параметрами по умолчанию...")

try
    crash_time, msg, mon = run_single(N_DEFAULT, S_DEFAULT, R_DEFAULT)
    println(msg)
    println("Время до падения: $(round(crash_time, digits=2)) часов")

```

```

        plot_results(mon, crash_time, N_DEFAULT, S_DEFAULT, R_DEFAULT)
    catch e
        println("Ошибка при запуске: $e")
    end

    println("\n" * "="^60)
    println("МАССОВЫЕ ПРОГОНЫ (RUNS=$RUNS)")
    println("="^60)

    results = run_experiments()

    println("\nРЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ:")
    println("-"^80)
    show(results, allcols=true, eltypes=false)

    if !isdir("results")
        mkdir("results")
    end

    CSV.write(datadir("ross", "ross_results.csv"), results)
    println("\n\nРезультаты сохранены ")

    txt_path = joinpath("results", "ross_results.txt")

```

```

open(txt_path, "w") do f
    write(f, "РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ МОДЕЛИ РОССА\n")
    write(f, "="^60 * "\n")
    write(f, "Параметры: RUNS=$RUNS\n\n")
    for row in eachrow(results)
        write(f, @sprintf("N=%2d, S=%2d, R=%2d: %8.2f ± %8.2f часов (min=%8.2f, max=%8.2f\n",
            row.N, row.S, row.R, row.mean_time, row.std_time, row.min_time, row.max_time)
    end
end

println("Текстовые результаты сохранены в $txt_path")

println("\n" * "="^60)
println("ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ")
println("="^60)
plot_heatmap(results)

compare_with_analytical()

analyze_repair_utilization()

println("\n" * "="^60)
println("АНАЛИЗ ЗАВЕРШЁН")
println("="^60)

return results

```

```
end
```

main (generic function with 1 method)

Запуск программы

```
if abspath(PROGRAM_FILE) == @__FILE__  
    main()  
end
```

5 Выводы

В ходе лабораторной работы были реализованы и исследованы дискретно-событийные модели $M/M/c$ и Росса. Для модели $M/M/c$ рассчитаны аналитические характеристики и построены соответствующие графики. Для модели Росса выполнены одиночный и массовые прогоны, проведено сравнение результатов симуляции с аналитическими значениями и исследована загрузка ремонтников. Результаты представлены в виде графиков и тепловых карт. Для обеих моделей также подготовлены литературные версии программ в форматах Quarto и Jupyter Notebook.